

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



DSA-Extended

Báo cáo bài tập lớn (Mở rộng)
PII-NER-ANONYMIZER

GVHD: Vương Bá Thịnh

Sinh viên thực hiện:	Trần Đình Ý	2414100
	Trần Văn Minh Triết	2413603
	Nguyễn Phúc Vĩnh	2414001
	Nguyễn Đình Minh	2412079

HỒ CHÍ MINH, THÁNG 5/ 2026

Tóm tắt nội dung

Trong thời đại số hiện nay, lượng dữ liệu văn bản chứa thông tin cá nhân ngày càng gia tăng và được sử dụng rộng rãi trong nhiều hệ thống khác nhau như mạng xã hội, thương mại điện tử, giáo dục và y tế. Điều này làm cho việc bảo vệ dữ liệu cá nhân trở thành một yêu cầu quan trọng trong các ứng dụng xử lý ngôn ngữ tự nhiên. Một trong những hướng tiếp cận phổ biến là sử dụng Named Entity Recognition (NER) để phát hiện và ẩn danh các thông tin định danh cá nhân trong văn bản.

Project này tập trung xây dựng một hệ thống nhận diện và ẩn danh thông tin cá nhân bằng phương pháp rule-based, không sử dụng machine learning hoặc deep learning. Hệ thống sử dụng các kỹ thuật xử lý chuỗi như regular expression, keyword matching, token scanning và context-based detection để phát hiện các thực thể quan trọng gồm NAME, EMAIL, PHONE, ADDRESS, USERNAME và URL. Sau khi phát hiện, các thông tin nhạy cảm sẽ được thay thế bằng các placeholder tương ứng nhằm giảm nguy cơ lộ dữ liệu cá nhân nhưng vẫn giữ được cấu trúc tổng quát của văn bản.

Pipeline của hệ thống bao gồm các bước preprocessing dữ liệu, chuyển đổi BIO labels thành ground truth entities, phát hiện entity bằng regex, phát hiện entity bằng ngữ cảnh, xử lý overlap giữa các entity, thực hiện anonymization và đánh giá kết quả bằng Precision, Recall và F1-score. Dataset được sử dụng trong project là [PII External Dataset](#) từ Kaggle, cung cấp dữ liệu văn bản cùng nhãn BIO phục vụ cho việc xây dựng ground truth và đánh giá kết quả của hệ thống.

Hệ thống cho thấy khả năng phát hiện tương đối tốt đối với các thực thể có cấu trúc cố định như email, URL và số điện thoại. Đồng thời, project cũng thể hiện vai trò của các thuật toán xử lý chuỗi trong bài toán NER và bảo vệ dữ liệu cá nhân. Mặc dù vẫn còn một số hạn chế về khả năng tổng quát hóa và xử lý các ngữ cảnh phức tạp, hệ thống đã xây dựng được một pipeline tương đối hoàn chỉnh cho bài toán phát hiện và ẩn danh thông tin cá nhân trong văn bản.

Từ khóa: Named Entity Recognition, PII Detection, Rule-based Method, Regular Expression, String Matching, Text Anonymization, Natural Language Processing.

Mục lục

Tóm tắt nội dung	i
Mục lục	ii
Danh sách hình vẽ	iv
Danh sách bảng	v
Danh sách đoạn mã	vi
1 Giới thiệu đề tài	1
2 Tổng quan mô hình và pipeline của hệ thống	2
3 Thiết kế module	3
3.1 Biểu thức chính quy (Regular expression - Regex)	3
3.2 Nhận diện thực thể bằng biểu thức chính quy (Regex Detector)	4
3.2.1 Nhận diện email	4
3.2.2 Nhận diện URL	4
3.2.3 Nhận diện số điện thoại	4
3.2.4 Nhận diện ngày tháng	5
3.2.5 Pipeline nội bộ và xử lý overlap	6
3.2.6 Minh họa hoạt động	7
3.3 Nhận diện thực thể theo ngữ cảnh (Context Detector)	7
3.3.1 Nhận diện tên người, tổ chức và địa điểm	8
3.3.2 Nhận diện địa chỉ	10
3.4 Tổng hợp kết quả	12
3.4.1 Vấn đề overlap	13
3.4.2 Hệ thống ưu tiên	13
3.4.3 Thuật toán Greedy Priority-First	13
3.4.4 Minh họa trường hợp overlap	14
4 Kiểm thử và đánh giá	15
4.1 Các chỉ số đánh giá	15
4.2 Kết quả thực nghiệm	16
4.3 Nhận xét	16

5	Tổng kết	17
6	Hướng mở rộng trong tương lai	18

Danh sách hình vẽ

2.1	Pipeline tổng quát của hệ thống	3
-----	---	---

Danh sách bảng

3.1	Kết quả phát hiện thực thể bằng regex detector	7
3.2	Danh sách thực thể trước khi hợp nhất	14
3.3	Danh sách thực thể sau khi hợp nhất	15
4.1	Kết quả đánh giá rule-based detector trên tập <code>pii_dataset.csv</code>	16

Danh sách đoạn mã

**PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ CỦA TỪNG
THÀNH VIÊN**

STT	HỌ VÀ TÊN	MSSV	NHIỆM VỤ	KẾT QUẢ
1	Trần Văn Minh Triết	2413603	Hiện thực module regex-detector.	100%
2	Nguyễn Đình Minh	2412079	Hiện thực module context-detector.	100%
3	Nguyễn Phúc Vĩnh	2414001	Thiết kế Web demo.	100%
4	Trần Đình Ý	2414100	Thực hiện merge, xử lý overlap và test.	100%

1 Giới thiệu đề tài

Named Entity Recognition, thường được viết tắt là NER, là một bài toán quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên. Mục tiêu chính của NER là phát hiện và phân loại các thực thể có ý nghĩa trong văn bản, chẳng hạn như tên người, địa điểm, tổ chức, email, số điện thoại, địa chỉ, tên tài khoản hoặc đường dẫn URL [1]. Thay vì chỉ xem văn bản như một chuỗi ký tự thông thường, NER giúp hệ thống hiểu được những thành phần nào trong văn bản mang thông tin quan trọng và chúng thuộc loại nào.

Trong thực tế, NER được ứng dụng trong nhiều bài toán khác nhau. Ví dụ, trong hệ thống tìm kiếm thông tin, NER giúp trích xuất tên người, địa danh hoặc tổ chức để cải thiện độ chính xác khi truy vấn. Trong lĩnh vực chăm sóc khách hàng, NER có thể dùng để nhận diện thông tin đơn hàng, địa chỉ giao hàng hoặc số điện thoại của người dùng. Trong các hệ thống quản lý tài liệu, NER hỗ trợ tự động phân loại và trích xuất thông tin quan trọng từ văn bản. Đặc biệt, đối với bài toán bảo vệ dữ liệu cá nhân, NER có vai trò phát hiện các thông tin định danh cá nhân, từ đó phục vụ cho việc ẩn danh hoặc che giấu dữ liệu nhạy cảm trước khi lưu trữ, chia sẻ hoặc xử lý tiếp.

Trong phạm vi project này, bài toán được tập trung vào việc nhận diện và ẩn danh thông tin cá nhân trong văn bản. Các loại thông tin cần phát hiện bao gồm NAME, EMAIL, PHONE, ADDRESS, USERNAME, ORGANIZATION, LOCATION, URL và DATE (optional - do nhóm đề xuất). Đây là những dạng dữ liệu có khả năng định danh trực tiếp hoặc gián tiếp một cá nhân. Ví dụ, một đoạn văn bản như:

My name is John Smith and my email is john@gmail.com.

sau khi xử lý có thể được chuyển thành:

My name is [NAME] and my email is [EMAIL].

Cách xử lý này giúp giảm nguy cơ lộ thông tin cá nhân, đồng thời vẫn giữ lại cấu trúc tổng quát của văn bản để phục vụ cho các bước phân tích tiếp theo.

Có nhiều hướng tiếp cận khác nhau để giải quyết bài toán NER. Hướng thứ nhất là sử dụng các phương pháp học máy hoặc học sâu, trong đó mô hình được huấn luyện trên dữ liệu đã gán nhãn để tự học cách nhận diện thực thể. Các mô hình hiện đại có thể đạt độ chính xác cao, đặc biệt khi có lượng dữ liệu lớn và tài nguyên tính toán phù hợp. Tuy nhiên, hướng tiếp cận này thường yêu cầu quá trình huấn luyện phức tạp, phụ thuộc nhiều vào dữ liệu, và khó giải thích rõ từng quyết định của hệ thống.

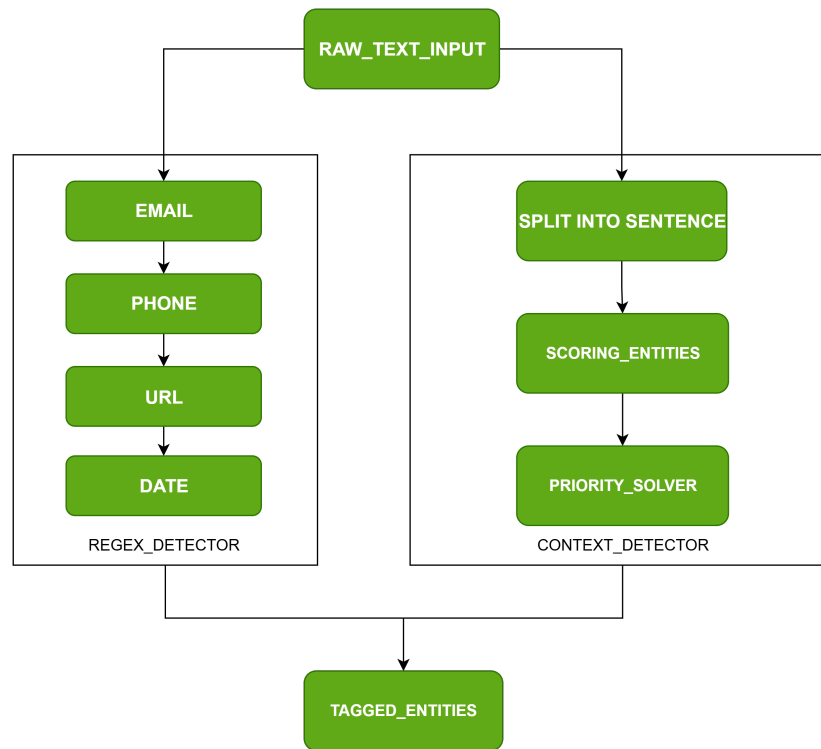
Hướng thứ hai là sử dụng phương pháp rule-based, tức là xây dựng các luật thủ công dựa trên đặc điểm chuỗi, biểu thức chính quy và ngữ cảnh xuất hiện của thực thể. Ví

dụ, email thường có ký tự @ và tên miền phía sau, URL thường bắt đầu bằng `http://`, `https://` hoặc `www.`, còn số điện thoại thường là chuỗi chữ số có độ dài nhất định và có thể chứa dấu cách, dấu gạch ngang hoặc mã quốc gia. Đối với các thực thể khó hơn như tên người hoặc địa chỉ, hệ thống có thể dựa vào các từ khóa ngữ cảnh như `my name is`, `full name`, `address`, `street`, `road`, `city`, v.v.

Trong project này, nhóm lựa chọn hướng rule-based thay vì machine learning hoặc deep learning. Lý do là project tập trung vào việc áp dụng các thuật toán xử lý chuỗi, regular expression, keyword matching, token scanning và xử lý khoảng giao nhau giữa các entity. Cách tiếp cận này phù hợp với mục tiêu học thuật vì dễ hiện thực, dễ kiểm soát, dễ giải thích và không cần quá trình huấn luyện mô hình. Ngoài ra, rule-based method cũng cho phép từng thành viên trong nhóm phát triển các module độc lập, sau đó tích hợp lại thành một pipeline hoàn chỉnh.

2 Tổng quan mô hình và pipeline của hệ thống

Pipeline tổng quát của hệ thống gồm các bước chính. Đầu tiên, dữ liệu thô được đọc từ file CSV và xử lý ở bước preprocessing. Trong bước này, các token và nhãn BIO trong dataset được chuyển thành danh sách ground truth entities để phục vụ đánh giá. Tiếp theo, hệ thống sử dụng regex-based detector để nhận diện các thực thể có cấu trúc rõ ràng như email, số điện thoại và URL. Sau đó, context-based detector được dùng để nhận diện các thực thể phụ thuộc nhiều vào ngữ cảnh như tên người, tổ chức, địa điểm, địa chỉ và username. Kết quả từ hai detector được đưa vào module merger để gộp, loại bỏ trùng lặp và xử lý các entity bị chồng lấn. Cuối cùng, hệ thống thực hiện ẩn danh văn bản và đánh giá kết quả bằng các chỉ số Precision, Recall và F1-score.



Hình 2.1: Pipeline tổng quát của hệ thống

3 Thiết kế module

3.1 Biểu thức chính quy (Regular expression - Regex)

Biểu thức chính quy trên một bảng chữ cái cho trước Σ là cách mô tả một tập chuỗi bằng cách kết hợp các thành phần cơ bản sau:[2]

- Các chuỗi ký hiệu được tạo từ bảng chữ cái Σ .
- Dấu ngoặc $(,)$ để nhóm biểu thức.
- Các toán tử hợp $+$, nối chuỗi $\cdot$, và bao đóng Kleene $*$.

Về mặt hình thức, một biểu thức chính quy trên bảng chữ cái Σ được xác định đệ quy bởi các điều kiện:

1. $\emptyset$, ϵ và mỗi ký hiệu $a \in \Sigma$ đều là biểu thức chính quy nguyên thủy.
 2. Nếu r và s là các biểu thức chính quy, thì (r) , $r + s$, $r \cdot s$ và r^* cũng là các biểu thức chính quy.
 3. Một chuỗi là biểu thức chính quy khi và chỉ khi nó có thể được suy ra từ các biểu thức chính quy nguyên thủy bằng một số hữu hạn lần áp dụng các quy tắc ở trên.
- Trong ngữ cảnh project, regex được dùng như một công cụ đặc tả pattern: thay vì liệt

kê từng chuỗi có thể xuất hiện, ta mô tả cấu trúc chung của cả một lớp chuỗi, chẳng hạn email, URL, số điện thoại hoặc ngày tháng.

3.2 Nhận diện thực thể bằng biểu thức chính quy (Regex Detector)

Module `regex_detector` chịu trách nhiệm nhận diện bốn loại thực thể có cấu trúc ký tự rõ ràng: `EMAIL`, `URL`, `PHONE` và `DATE`. Đây là các thực thể mà mẫu chuỗi tương đối cố định, phù hợp với phương pháp regular expression. Module trả về danh sách entity theo đúng schema chung của project: `type`, `text`, `start`, `end`.

Về mặt thiết kế, module được tổ chức theo pipeline nội bộ: phát hiện email và URL trước, sau đó dùng kết quả này làm exclusion spans khi phát hiện phone, rồi dùng kết quả email, URL và phone làm exclusion spans khi phát hiện date. Cách tiếp cận này giúp giảm thiểu false positive giữa các entity cùng module.

3.2.1 Nhận diện email

Biểu thức chính quy cho email tuân theo cấu trúc `local@domain.tld`:

- **Local part**: gồm chữ cái, chữ số, dấu chấm, dấu cộng, gạch dưới, gạch ngang.
- **Domain**: một hoặc nhiều nhãn phân cách bằng dấu chấm, mỗi nhãn chứa chữ cái hoặc chữ số.
- **TLD**: ít nhất 2 ký tự chữ cái.

Sau khi regex match, hệ thống loại bỏ dấu câu cuối (trailing punctuation) như dấu chấm, dấu phẩy, dấu chấm than — vì các ký tự này thường thuộc ngữ pháp câu, không phải phần email.

3.2.2 Nhận diện URL

Pattern URL bắt đầu bằng `http://`, `https://` hoặc `www.` và lấy tất cả ký tự liên tiếp cho đến khoảng trắng. Sau đó áp dụng hai bước hậu xử lý:

1. Loại bỏ trailing punctuation (dấu chấm, dấu phẩy, v.v.).
2. Cân bằng dấu ngoặc đơn: nếu URL kết thúc bằng `)` nhưng không chứa `(` bên trong, dấu `)` được coi là thuộc ngữ pháp câu và bị loại bỏ.

3.2.3 Nhận diện số điện thoại

Regex cho phone bao phủ nhiều định dạng phổ biến:

- Chuỗi liên tục 8–15 chữ số (0912345678).
 - Định dạng có một separator (0475 4429797).
 - Mã quốc gia tùy chọn, mã vùng trong ngoặc, các nhóm số phân cách bằng khoảng trắng, dấu chấm hoặc gạch ngang (+84 912 345 678, (091) 234 5678, 415.555.2671).
- Regex phone có phạm vi rộng nên cần hậu xử lý nghiêm ngặt để giảm false positive:
1. **Exclusion:** bỏ qua candidate nếu span chồng lấn với entity email hoặc URL đã phát hiện trước đó.
 2. **Trim:** loại bỏ khoảng trắng đầu/cuối và trailing punctuation.
 3. **Kiểm tra số chữ số:** tổng số digit phải nằm trong khoảng [9, 15]. Điều kiện này loại bỏ các số ngắn (mã bưu điện, năm) và số quá dài.
 4. **Boundary check:** ký tự ngay trước **start** và ngay sau **end** không được là chữ cái, chữ số, hoặc @, nhằm tránh bắt nhầm một phần của token lớn hơn.

3.2.4 Nhận diện ngày tháng

Entity DATE bao gồm nhiều định dạng ngày tháng phổ biến trong tiếng Anh và tiếng Việt. Hệ thống sử dụng 11 biểu thức chính quy, chia thành bốn nhóm.

Nhóm 1: Định dạng tiếng Việt

- ngày DD tháng MM năm YYYY — có hoặc không có prefix “ngày”.
- ngày D/M/YYYY — prefix “ngày” kèm dấu gạch chéo.
- tháng MM năm YYYY — chỉ tháng/năm.
- năm YYYY — chỉ năm, yêu cầu prefix “năm” để tránh false positive.

Nhóm 2: Định dạng tên tháng tiếng Anh

- January 2, 2024 / Jan 2, 2024 — tên tháng đầy đủ hoặc viết tắt, theo sau là ngày và năm.
- 2 January 2024 / 2 Jan 2024 — ngày đứng trước tên tháng.
- January 2024 / Jan 2024 — chỉ tháng/năm.
- year YYYY — chỉ năm, yêu cầu prefix “year”.

Nhóm 3: Định dạng số (ISO và phổ biến)

- YYYY-MM-DD / YYYY/MM/DD — ISO 8601.
- DD/MM/YYYY, DD-MM-YYYY, DD.MM.YYYY — hỗ trợ cả năm 2 và 4 chữ số.
- MM/YYYY — tháng/năm dạng số.

Nhóm 4: Năm đơn lẻ có ngữ cảnh

Số 4 chữ số đứng một mình (ví dụ 2024) **không** được nhận diện là DATE, vì rất dễ trùng với mã số, chỉ số hoặc số lượng. Chỉ khi có prefix rõ ràng như năm hoặc year, hệ thống mới phát hiện.

Giảm false positive cho DATE

1. **Exclusion spans:** candidate DATE bị loại nếu chồng lấn với bất kỳ entity EMAIL, URL hoặc PHONE đã phát hiện trước đó. Điều này ngăn việc bắt nhầm segment giống ngày trong URL (ví dụ `https://example.com/2024/01/02`).
2. **De-duplication nội bộ:** khi nhiều pattern cùng match overlapping span (ví dụ pattern DD/MM/YYYY và MM/YYYY cùng match trên chuỗi 01/02/2024), hệ thống giữ match dài nhất.
3. **Lookbehind/lookahead:** các pattern số dùng negative lookbehind (`?<!\d`) và negative lookahead (`?!\d`) để đảm bảo không bắt nhầm một phần của số dài hơn.

3.2.5 Pipeline nội bộ và xử lý overlap

Hàm chính `detect_regex_entities` gọi bốn detector theo thứ tự phụ thuộc, truyền kết quả đã phát hiện làm exclusion cho detector tiếp theo:

1. Phát hiện EMAIL $\rightarrow$ danh sách E_{email} .
2. Phát hiện URL $\rightarrow$ danh sách E_{url} .
3. Phát hiện PHONE với exclusion $= E_{email} \cup E_{url} \rightarrow$ danh sách E_{phone} .
4. Phát hiện DATE với exclusion $= E_{email} \cup E_{url} \cup E_{phone} \rightarrow$ danh sách E_{date} .

Sau khi có đủ bốn danh sách, tất cả entity được gộp lại và đưa qua hàm `_resolve_regex_overlaps` để xử lý overlap cuối cùng theo priority:

$$\text{EMAIL} > \text{URL} > \text{PHONE} > \text{DATE}$$

Thuật toán giải quyết overlap giống với merger tổng thể (Greedy Priority-First): sắp xếp theo priority rồi duyệt tham lam, giữ entity nào không chồng lấn với các entity đã giữ trước đó.

Algorithm 1 Pipeline nhận diện thực thể bằng regex

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể E không overlap, sắp theo vị trí

- 1: $E_{email} \leftarrow \text{DETECTEMAIL}(T)$
 - 2: $E_{url} \leftarrow \text{DETECTURL}(T)$
 - 3: $higher \leftarrow E_{email} \cup E_{url}$
 - 4: $E_{phone} \leftarrow \text{DETECTPHONE}(T, higher)$
 - 5: $E_{date} \leftarrow \text{DETECTDATE}(T, higher \cup E_{phone})$
 - 6: $all \leftarrow E_{email} \cup E_{url} \cup E_{phone} \cup E_{date}$
 - 7: $E \leftarrow \text{RESOLVEREGEXOVERLAPS}(all)$
 - 8: **return** E
-

3.2.6 Minh họa hoạt động

Xét đoạn văn bản:

“Email john@a.com, born 01/02/2024, call +84 912 345 678.”

Kết quả phát hiện của module được trình bày trong Bảng 3.1.

Bảng 3.1: Kết quả phát hiện thực thể bằng regex detector

Loại	Text	start	end
EMAIL	john@a.com	6	16
DATE	01/02/2024	23	33
PHONE	+84 912 345 678	40	55

Trong ví dụ trên, ba entity không chồng lấn nên tất cả đều được giữ lại. Nếu có URL chứa segment giống ngày (ví dụ <https://example.com/2024/01/02>), segment đó sẽ bị loại nhờ exclusion spans và chỉ URL được giữ.

3.3 Nhận diện thực thể theo ngữ cảnh (Context Detector)

Thành phần nhận diện theo ngữ cảnh nhận đầu vào là một đoạn văn bản thô và trả về danh sách các thực thể có dạng **type**, **text**, **start**, **end**. Khác với nhóm nhận diện

bằng biểu thức chính quy, nhóm này không chỉ dựa vào cấu trúc ký tự cố định mà còn khai thác các từ khóa, vị trí xuất hiện, chữ hoa và quan hệ giữa các từ trong cùng câu.

Các loại thực thể được xử lý gồm tên người (NAME), tổ chức (ORGANIZATION), địa điểm (LOCATION), tên người dùng (USERNAME) và địa chỉ (ADDRESS). Về mặt thuật toán, quá trình nhận diện được tổ chức theo luồng: chuẩn hóa ngữ cảnh đầu vào, sinh candidate, chấm điểm hoặc kiểm tra mẫu, sau đó trả về các span thực thể không chồng lấn.

3.3.1 Nhận diện tên người, tổ chức và địa điểm

Đầu vào của thuật toán là văn bản đã được chia thành các câu cùng với vị trí của từng câu trong văn bản gốc. Đầu ra là các span được gán một trong ba nhãn: NAME, ORGANIZATION hoặc LOCATION. Ba loại thực thể này đều có thể xuất hiện dưới dạng cụm danh từ riêng viết hoa, vì vậy hệ thống không chạy bộ phát hiện tách rời. Thay vào đó, thuật toán sinh candidate một lần, chấm điểm candidate đó theo cả ba nhãn, rồi quyết định nhãn cuối cùng ở bước sau cùng.

Trước khi phân tích ngữ cảnh, văn bản đầu vào được tách thành các câu riêng lẻ. Đây là bước quan trọng vì điểm ngữ cảnh (pronoun, nghề nghiệp, trigger) chỉ có ý nghĩa trong phạm vi một câu.

Thách thức chính là phân biệt dấu chấm kết thúc câu với dấu chấm trong các trường hợp khác. Thuật toán tách câu xử lý điều này theo quy tắc: dấu . chỉ được coi là ranh giới câu khi *đồng thời* thỏa ba điều kiện:

1. Token kết thúc bằng . không thuộc danh sách viết tắt đã biết (Dr., Mr., St., Apt., Inc., v.v.).
 2. Phần văn bản ngay sau . không bắt đầu bằng tên miền cấp cao (com, edu, vn, org, v.v.) — nhằm tránh split nhầm địa chỉ email và URL.
 3. Sau dấu . là khoảng trắng hoặc hết chuỗi.
- Dấu ! và ? luôn được coi là kết thúc câu vô điều kiện.

Bước 1: xử lý trigger mạnh cho tên người

Thuật toán trước hết tìm các cụm từ báo hiệu trực tiếp rằng phần đứng sau là tên người. Các trigger này không tạo ra một luồng nhận diện riêng, mà được lưu như một tín hiệu điểm mạnh cho nhãn NAME. Trigger được chia thành hai nhóm:

- **NAME_TRIGGERS**: cụm từ chỉ thẳng tên chủ thể, gồm "my name is", "full name", "name:".
- **_LABEL_TRIGGERS**: nhãn hay đứng trước tên trong văn bản có cấu trúc, gồm khoảng 25 mẫu như "contact information:", "author:", "submitted by:",

"regards,", "sincerely,", v.v.

Khi tìm thấy trigger, thuật toán thu thập 1–4 token liên tiếp thỏa: bắt đầu bằng chữ hoa, không chứa chữ số, không chứa ký tự đặc biệt (ngoại trừ dấu gạch ngang – vì tên có thể có dạng **Mary-Jane**), không phải stopword, và không mang đuôi động từ sau token đầu tiên (-ing, -tion, -ed, v.v.). Span tìm được sẽ nhận điểm **NAME** rất cao nếu trùng với candidate ở bước chấm điểm.

Bước 2: tìm candidate bằng linear search và rule-based filtering

Với mỗi câu, thuật toán sinh các candidate danh từ riêng từ những cụm token viết hoa. Candidate được dùng chung cho cả ba nhãn **NAME**, **ORGANIZATION** và **LOCATION**, thay vì viết lại ba hàm quét riêng.

Một candidate hợp lệ phải bắt đầu bằng chữ hoa, không chứa chữ số, không chứa ký tự đặc biệt, không phải stopword, không phải đại từ, không phải từ mở đầu câu thường gặp, không phải title prefix và không phải chức danh công việc. Thuật toán cũng cho phép một số connector như **of**, **the**, **and** để bắt các cụm như **University of California**.

Bước 3: scoring candidate theo ba nhãn

Chương trình sử dụng thuật toán Heuristic Rule-Based Scoring để gán điểm cho mỗi candidate theo ba nhãn. Mỗi nhãn có một tập hợp các quy tắc riêng. Ví dụ: tần suất xuất hiện của các từ đặc biệt, cấu trúc cố định,...

Output

Sau khi có đủ ba điểm, thuật toán chọn đúng một nhãn cuối cùng cho candidate. Nhãn có điểm cao nhất và đạt ngưỡng sẽ được giữ. Nếu nhiều nhãn bằng điểm, thuật toán dùng thứ tự tie-break ổn định để tránh một span được xuất ra nhiều lần với nhiều tag khác nhau. Vì vậy, một span như **OpenAI** trong câu **I work at OpenAI**. được quyết định là **ORGANIZATION**, không đồng thời xuất hiện thêm dưới dạng **NAME** hoặc **LOCATION**.

Sau khi mỗi candidate đã có nhãn cuối cùng, hệ thống loại trùng và overlap theo priority nội bộ, điểm, vị trí bắt đầu và độ dài span trong phạm vi các kết quả do context detector sinh ra. Kết quả này vẫn có thể được đưa sang merger tổng thể để xử lý overlap với các thực thể từ regex detector.

Algorithm 2 Thuật toán nhận diện NAME, ORGANIZATION và LOCATION

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể E

```
1:  $E \leftarrow \emptyset$ 
2:  $lower \leftarrow \text{lowercase}(T)$ 
// Bước 1: Ghi nhận trigger mạnh cho NAME
3: for mỗi trigger  $tr$  trong  $\text{NAMETRIGGERS} \cup \text{LABELTRIGGERS}$  do
4:   for mỗi vị trí  $pos$  của  $tr$  trong  $lower$  do
5:     Thu thập 1–4 token hoa liên tiếp sau  $pos$  vào  $tokens$ 
6:     if  $tokens \neq \emptyset$  then
7:       Lưu span  $\text{join}(tokens)$  vào  $strongNameSpans$  với điểm 100
8:     end if
9:   end for
10: end for
// Bước 2–4: Candidate chung, scoring ba nhãn, chọn tag
11:  $sentences \leftarrow \text{SPLITSENTENCES}(T)$ 
12: for mỗi câu  $s$  trong  $sentences$  do
13:   for mỗi candidate  $c$  trong  $\text{EXTRACTENTITYCANDIDATES}(s)$  do
14:      $nameScore \leftarrow \text{SCORENAME}(c, s, strongNameSpans)$ 
15:      $orgScore \leftarrow \text{SCOREORGANIZATION}(c, s)$ 
16:      $locScore \leftarrow \text{SCORELOCATION}(c, s)$ 
17:      $tag \leftarrow \text{CHOOSEBESTTAG}(nameScore, orgScore, locScore)$ 
18:     if  $tag$  tồn tại then
19:       Thêm  $c$  vào  $E$  với nhãn  $tag$  và điểm tương ứng
20:     end if
21:   end for
22: end for
23: return  $\text{RESOLVEENTITIES}(E)$ 
```

3.3.2 Nhận diện địa chỉ

Đầu vào của thuật toán nhận diện địa chỉ là văn bản gốc và danh sách span email đã được xác định trước. Đầu ra là các span ADDRESS. Địa chỉ trong tập dữ liệu thường tuân theo cấu trúc gồm số nhà, tên đường, loại đường, và hậu tố hướng hoặc đơn vị căn hộ tùy chọn. Thuật toán sử dụng hai cơ chế bổ sung cho nhau.

Bước 1: nhận diện địa chỉ bằng Regex pattern

Biểu thức chính quy `_ADDRESS_RE` mô tả cấu trúc địa chỉ theo bốn thành phần nối tiếp:

1. **Số nhà:** `\b(\d{1,5})(?!\\d)\\s+` — từ 1 đến 5 chữ số, kèm negative lookahead `(?!\\d)` để đảm bảo đây không phải là một phần của số dài hơn (loại trừ số điện thoại, mã bưu điện dài).
2. **Tên đường:** `((?:[A-Z0-9][A-Za-z0-9]*\\s+)+?)` — một hoặc nhiều từ bắt đầu bằng chữ hoa hoặc chữ số, sử dụng non-greedy để nhường ưu tiên cho thành phần loại đường phía sau.
3. **Loại đường:** danh sách khoảng 30 từ được sắp xếp từ dài đến ngắn để tránh match sớm, bao gồm dạng đầy đủ (Boulevard, Terrace, Avenue, Street, Drive, Road, v.v.) và dạng viết tắt (Blvd., Ave., Rd., St., v.v.).
4. **Hướng và đơn vị** (tùy chọn): hướng gồm tám giá trị North, South, East, West và các tổ hợp; đơn vị gồm Suite, Apt., Apartment, Unit kèm số phòng.

Trước khi kiểm tra kết quả regex, tất cả span email trong văn bản được xác định trước; bất kỳ span địa chỉ nào nằm hoàn toàn bên trong span email sẽ bị bỏ qua để tránh nhầm lẫn phần tên miền của địa chỉ email với địa chỉ nhà.

Bước 2: nhận diện địa chỉ bằng trigger keyword

Để bổ sung cho regex chính, đặc biệt với các địa chỉ được viết theo dạng nhân hoặc clause theo trigger, module sử dụng danh sách trigger bao gồm **street, road, avenue, district, city, st., rd., ave..** Khi tìm thấy trigger, thuật toán chỉ tiếp tục nếu ngay sau trigger có dấu : hoặc cụm **is**. Sau đó hệ thống lấy clause đến dấu câu gần nhất, loại bỏ noise words đầu clause (**at, is, the, located, live, reside, v.v.**), và kiểm tra thêm: số nhà ở đầu clause không được vượt quá 5 chữ số, span không nằm trong email, và span không được trùng lặp với kết quả từ cơ chế regex.

Algorithm 3 Thuật toán nhận diện địa chỉ

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể địa chỉ E

```
1:  $E \leftarrow \emptyset$ 
2:  $emailSpans \leftarrow \text{FINDEMAILSPANS}(T)$ 
// Cơ chế 1: Regex pattern
3: for mỗi match  $m$  của ADDRESSRE trong  $T$  do
4:   if  $m$  nằm trong một email span then continue
5:   end if
6:   Thêm span của  $m$  vào  $E$ 
7: end for
// Cơ chế 2: Trigger keyword
8: for mỗi trigger  $tr$  trong ADDRESSTRIGGERS do
9:   for mỗi vị trí  $pos$  của  $tr$  trong  $T$  do
10:    if sau  $tr$  không phải : hoặc is then continue
11:    end if
12:     $clause \leftarrow \text{EXTRACTCLAUSE}(T, pos)$ 
13:     $addr \leftarrow \text{STRIPLEADINGNOISE}(clause)$ 
14:    if số nhà đầu  $addr$  có nhiều hơn 5 chữ số then continue
15:    end if
16:    if  $addr$  nằm trong email span then continue
17:    end if
18:    if  $addr$  overlap với span đã có trong  $E$  then continue
19:    end if
20:    Thêm span của  $addr$  vào  $E$ 
21:  end for
22: end for
23: return DEDUPLICATE( $E$ )
```

3.4 Tổng hợp kết quả

Đầu vào của bước hợp nhất là nhiều danh sách thực thể được sinh ra từ các nhóm nhận diện khác nhau. Đầu ra là một danh sách duy nhất, đã loại trùng và không còn các span chồng lấn. Thuật toán hợp nhất là bước quyết định kết quả cuối cùng trước khi văn bản được mã hoá.

3.4.1 Vấn đề overlap

Hai thực thể được coi là *chồng lấn* (overlap) nếu khoảng ký tự của chúng giao nhau, tức là $[s_1, e_1)$ và $[s_2, e_2)$ thỏa $s_1 < e_2$ và $e_1 > s_2$. Overlap xảy ra trong nhiều tình huống thực tế:

- Địa chỉ email `kazuosun@hotmail.net` bị `regex_detector` bắt đúng là **EMAIL**, trong khi các detector khác có thể tạo span chồng lấn trên một phần của chuỗi này.
- Một chuỗi như `123 Main Street` có thể được nhận là **ADDRESS**, đồng thời phần `Main Street` cũng có thể được nhận là **LOCATION**.
- Một cụm như `OpenAI Research Lab` có thể được chấm điểm theo nhiều nhãn ngữ cảnh; khi đó detector cần chọn một nhãn cuối cùng trước khi đưa kết quả sang bước hợp nhất.

3.4.2 Hệ thống ưu tiên

Để giải quyết overlap một cách nhất quán, merger định nghĩa một thứ tự ưu tiên tuyến tính cho tám loại thực thể:

EMAIL > URL > PHONE > ADDRESS > ORGANIZATION > LOCATION > NAME > USERNAME

Thứ tự này phản ánh độ tin cậy của từng detector: **EMAIL**, **URL** và **PHONE** được phát hiện bằng regex có độ chính xác cao và cấu trúc rõ ràng nên được ưu tiên hơn; **ADDRESS** thắng **LOCATION** để tránh mất địa chỉ đầy đủ; **ORGANIZATION**, **LOCATION**, **NAME** và **USERNAME** dựa nhiều hơn vào ngữ cảnh nên được xếp sau các entity có cấu trúc rõ.

3.4.3 Thuật toán Greedy Priority-First

Thuật toán hợp nhất thực hiện giải quyết overlap theo chiến lược tham lam ưu tiên (greedy priority-first):

1. Gộp tất cả thực thể từ mọi detector thành một danh sách duy nhất.
2. Sắp xếp danh sách theo ba tiêu chí lần lượt: (a) thứ hạng ưu tiên tăng dần (loại ưu tiên cao xếp trước), (b) chỉ số **start** tăng dần, (c) độ dài span giảm dần (span dài hơn xếp trước khi cùng điểm xuất phát).
3. Duyệt tuần tự qua danh sách đã sắp xếp. Với mỗi thực thể, giữ lại nếu span của nó không chồng lấn với bất kỳ thực thể đã giữ nào; bỏ qua nếu ngược lại.
4. Sắp xếp lại danh sách kết quả theo **start** để đảm bảo thứ tự xuất hiện trong văn bản.

Algorithm 4 Thuật toán hợp nhất và giải quyết overlap (Greedy Priority-First)

Require: Các nhóm thực thể $G_1, G_2, \dots, G_k$ từ các detector

Ensure: Danh sách thực thể không overlap, sắp theo vị trí

```
1:  $all \leftarrow G_1 \cup G_2 \cup \dots \cup G_k$ 
2: Sắp xếp  $all$  theo ( $priority(type)$ ,  $start$ ,  $-(end - start)$ ) tăng dần
3:  $kept \leftarrow \emptyset$ 
4:  $occupied \leftarrow \emptyset$  // Tập các span  $(s, e)$  đã giữ
5: for mỗi thực thể  $ent$  trong  $all$  do
6:    $s \leftarrow ent.start$ ,  $e \leftarrow ent.end$ 
7:   if  $\nexists (os, oe) \in occupied$  sao cho  $s < oe$  và  $e > os$  then
8:      $kept \leftarrow kept \cup \{ent\}$ 
9:      $occupied \leftarrow occupied \cup \{(s, e)\}$ 
10:  end if
11: end for
12: Sắp xếp  $kept$  theo  $start$  tăng dần
13: return  $kept$ 
```

3.4.4 Minh họa trường hợp overlap

Xét đoạn văn bản sau:

"I live at 123 Main Street."

Trong đoạn này, ADDRESS và LOCATION có thể cùng nhận diện vùng văn bản liên quan đến Main Street. Danh sách trước khi hợp nhất có dạng như Bảng 3.2.

Bảng 3.2: Danh sách thực thể trước khi hợp nhất

Detector	Loại	Text	Ưu tiên
context_detector	ADDRESS	123 Main Street	4
context_detector	LOCATION	Main Street	6

Sau khi sắp xếp theo ưu tiên và áp dụng thuật toán greedy, merger giữ 123 Main Street vì ADDRESS có priority cao hơn LOCATION. Candidate Main Street bị loại do overlap với span địa chỉ đầy đủ đã được giữ. Kết quả cuối cùng được trình bày trong Bảng 3.3.

Bảng 3.3: Danh sách thực thể sau khi hợp nhất

Loại	Text	Vị trí
ADDRESS	123 Main Street	start=10, end=25

Kết quả này sau đó được đưa vào module `anonymizer` để thay thế các span PII bằng placeholder tương ứng, tạo ra văn bản đã được ẩn danh hóa.

4 Kiểm thử và đánh giá

Phần kiểm thử tập trung đánh giá độ chính xác của thuật toán rule-based trên tập dữ liệu gốc `data/raw/pii_dataset.csv`. Script kiểm thử được đặt tại `tests/test_rule_based_accuracy.py`. Với mỗi dòng dữ liệu, hệ thống lấy trường `text` làm đầu vào, chạy pipeline phát hiện thực thể hiện tại gồm `regex_detector`, `context_detector` và `merger`, sau đó so sánh kết quả nhận diện với các cột ground truth trong file CSV.

Các loại thực thể được đánh giá trong phần này gồm:

NAME, EMAIL, PHONE, URL

Trong đó, NAME được so sánh với cột `name`, EMAIL với cột `email`, PHONE với cột `phone`, và URL với cột `url`. Đối với số điện thoại, script chuẩn hóa về chuỗi chữ số trước khi so sánh nhằm tránh sai lệch do dấu cách, dấu ngoặc hoặc dấu gạch ngang.

4.1 Các chỉ số đánh giá

Script in ra các chỉ số sau:

- **Support**: số dòng có giá trị ground truth cho loại thực thể tương ứng.
- **Correct**: số dòng mà detector tìm được đúng giá trị ground truth trong danh sách dự đoán.
- **Accuracy**: tỉ lệ $\text{Correct} / \text{Support}$. Chỉ số này cho biết trong các dòng có ground truth, hệ thống có tìm được đúng thực thể cần tìm hay không.
- **Precision**: tỉ lệ $\text{TP} / (\text{TP} + \text{FP})$. Chỉ số này cho biết trong các thực thể hệ thống dự đoán ra, bao nhiêu thực thể là đúng.
- **Recall**: tỉ lệ $\text{TP} / (\text{TP} + \text{FN})$, tương đương khả năng tìm lại đúng các thực thể có trong ground truth.
- **F1**: trung bình điều hòa giữa precision và recall.

Vì mỗi dòng CSV chỉ có một giá trị ground truth cho từng cột như `name` hoặc `email`, **Accuracy** trong bảng dưới đây được hiểu là tỉ lệ tìm đúng giá trị chính của dòng đó. Chỉ số này khác với **Precision**: một detector có thể tìm đúng tên thật trong dòng văn bản nhưng đồng thời dự đoán dư nhiều cụm khác là **NAME**; khi đó accuracy vẫn cao nhưng precision thấp.

4.2 Kết quả thực nghiệm

Kết quả được ghi nhận khi chạy lệnh:

```
python tests/test_rule_based_accuracy.py
```

Tổng số dòng được đánh giá là 4434. Kết quả chi tiết được trình bày trong Bảng 4.1.

Bảng 4.1: Kết quả đánh giá rule-based detector trên tập `pii_dataset.csv`

Entity	Support	Correct	Accuracy	Precision	Recall	F1	FP	FN
NAME	4434	3845	86.72%	18.39%	86.72%	30.35%	17060	589
EMAIL	4434	3825	86.27%	95.79%	86.27%	90.78%	168	609
PHONE	4434	2383	53.74%	99.04%	53.74%	69.68%	23	2051
URL	1322	526	39.79%	86.23%	39.79%	54.45%	84	796
Overall	14624	10579	72.34%	37.90%	72.34%	49.74%	17335	4045

4.3 Nhận xét

Kết quả cho thấy nhóm thực thể có cấu trúc rõ ràng như **EMAIL** đạt hiệu quả tốt nhất. Precision của **EMAIL** đạt 95.79%, cho thấy phần lớn email được detector dự đoán ra là chính xác. Tuy nhiên recall chỉ đạt 86.27%, nghĩa là vẫn còn một số email trong dataset chưa được phát hiện, thường do khác biệt format hoặc dấu câu đi kèm trong văn bản.

Đối với **PHONE**, precision rất cao, đạt 99.04%, nhưng recall chỉ đạt 53.74%. Điều này cho thấy regex số điện thoại khá thận trọng: khi đã nhận diện thì thường đúng, nhưng bỏ sót nhiều trường hợp có định dạng đa dạng. Đây là một điểm cần cải thiện nếu muốn tăng độ phủ của hệ thống.

URL có precision tương đối cao, đạt 86.23%, nhưng accuracy và recall chỉ đạt 39.79%. Nguyên nhân chính là dataset có nhiều URL hoặc placeholder URL không theo đúng dạng

mà regex hiện tại ưu tiên, chẳng hạn các dạng thiếu scheme hoặc bị gắn với dấu câu trong câu.

Với **NAME**, accuracy đạt 86.72%, nghĩa là hệ thống thường tìm được tên chính của dòng dữ liệu. Tuy nhiên precision chỉ đạt 18.39% do detector theo ngữ cảnh còn dự đoán dư nhiều cụm viết hoa không phải tên người. Điều này phản ánh đặc thù của rule-based name detection: nếu rule rộng thì recall cao hơn nhưng dễ phát sinh false positive; nếu rule chặt hơn thì precision tăng nhưng có nguy cơ bỏ sót tên thật.

Nhìn chung, kết quả overall đạt accuracy 72.34% và F1 49.74%. Hệ thống rule-based hiện tại phù hợp với các thực thể có cấu trúc cố định, đặc biệt là **EMAIL** và các số điện thoại có format rõ ràng. Các thực thể phụ thuộc nhiều vào ngữ cảnh hoặc có format đa dạng như **NAME** và **URL** cần được bổ sung thêm rule, dictionary hoặc cơ chế lọc false positive để cải thiện precision và recall.

5 Tổng kết

Trong project này, nhóm đã xây dựng một hệ thống nhận diện và ẩn danh thông tin cá nhân trong văn bản bằng phương pháp rule-based. Hệ thống tập trung vào việc phát hiện các thực thể quan trọng như **NAME**, **EMAIL**, **PHONE**, **ADDRESS**, **USERNAME** và **URL**. Thay vì sử dụng các mô hình machine learning hoặc deep learning, project áp dụng các kỹ thuật xử lý chuỗi như regular expression, keyword matching, token scanning và xử lý overlap giữa các entity để giải quyết bài toán NER.

Trong quá trình thực hiện, nhóm đã xây dựng được một pipeline xử lý tương đối hoàn chỉnh, bao gồm các bước preprocessing dữ liệu, phát hiện entity bằng regex, phát hiện entity bằng ngữ cảnh, gộp kết quả, xử lý entity chồng lấn, ẩn danh văn bản và đánh giá kết quả bằng các chỉ số Precision, Recall và F1-score. Việc chia hệ thống thành nhiều module độc lập giúp các thành viên có thể phát triển song song, đồng thời hỗ trợ quá trình kiểm thử và mở rộng sau này trở nên dễ dàng hơn.

Kết quả đạt được cho thấy phương pháp rule-based hoạt động tương đối hiệu quả đối với các thực thể có cấu trúc rõ ràng như email, URL và số điện thoại. Các loại thực thể này thường có format tương đối cố định nên phù hợp với phương pháp regular expression. Đối với các thực thể phụ thuộc nhiều vào ngữ cảnh như tên người hoặc địa chỉ, hệ thống vẫn có thể phát hiện được bằng cách sử dụng các luật thủ công dựa trên keyword và context. Ngoài ra, cơ chế xử lý overlap giữa các entity cũng giúp giảm một phần lỗi nhận diện trùng lặp hoặc phát hiện sai trong văn bản.

Bên cạnh những kết quả đạt được, project vẫn còn tồn tại một số hạn chế nhất định.

Vì sử dụng rule-based method nên chất lượng nhận diện phụ thuộc rất nhiều vào tập luật được xây dựng thủ công. Nếu văn bản có cấu trúc quá đa dạng hoặc xuất hiện các format hiếm gặp, hệ thống có thể bỏ sót entity hoặc nhận diện sai. Đặc biệt, các thực thể như tên người và địa chỉ thường có rất nhiều biến thể khác nhau nên khó đạt độ chính xác cao chỉ bằng các luật đơn giản. Ngoài ra, phương pháp đánh giá exact match trong bài toán NER cũng khá nghiêm ngặt, chỉ cần lệch một phần nhỏ vị trí span thì prediction sẽ bị xem là sai hoàn toàn.

Mặc dù vậy, project vẫn thể hiện rõ vai trò quan trọng của bài toán NER trong thời đại AI hiện nay. Khi lượng dữ liệu văn bản ngày càng lớn và được sử dụng trong nhiều hệ thống khác nhau, nhu cầu bảo vệ thông tin cá nhân ngày càng trở nên cần thiết. Các kỹ thuật NER có thể hỗ trợ tự động phát hiện và ẩn danh dữ liệu nhạy cảm trước khi lưu trữ, chia sẻ hoặc xử lý tiếp. Điều này đặc biệt quan trọng trong các lĩnh vực như tài chính, y tế, giáo dục, thương mại điện tử và mạng xã hội, nơi dữ liệu cá nhân thường xuyên được xử lý với số lượng lớn.

Thông qua project này, nhóm không chỉ hiểu rõ hơn về bài toán nhận diện thực thể trong xử lý ngôn ngữ tự nhiên mà còn có cơ hội áp dụng các thuật toán xử lý chuỗi, regular expression và thiết kế pipeline xử lý dữ liệu thực tế. Đây cũng là nền tảng quan trọng để tiếp cận các hướng nghiên cứu hoặc hệ thống NER phức tạp hơn trong tương lai.

6 Hướng mở rộng trong tương lai

Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng khác nhau nhằm cải thiện độ chính xác, khả năng tổng quát hóa và khả năng ứng dụng thực tế của bài toán NER. Một trong những hướng phát triển quan trọng là bổ sung thêm rule và dictionary để tăng khả năng nhận diện tên riêng, địa danh và địa chỉ. Việc sử dụng danh sách first name, last name, tên đường hoặc địa danh phổ biến có thể giúp giảm false positive và tăng độ chính xác cho các context-based detector.

Ngoài ra, project hiện chủ yếu hoạt động trên dữ liệu tiếng Anh nên vẫn chưa tối ưu cho tiếng Việt. Trong tương lai, hệ thống có thể được mở rộng sang dữ liệu tiếng Việt bằng cách xây dựng thêm các rule phù hợp với cấu trúc tên người, số điện thoại và địa chỉ tại Việt Nam. Đây là hướng mở rộng có ý nghĩa thực tế cao vì nhiều hệ thống trong nước hiện vẫn gặp khó khăn trong việc xử lý dữ liệu tiếng Việt.

Một hướng cải tiến khác là tối ưu quá trình xử lý overlap giữa các entity. Hiện tại, project chủ yếu sử dụng cơ chế priority-based greedy selection để xử lý entity chồng

lần. Trong tương lai, có thể áp dụng các kỹ thuật như interval scheduling hoặc weighted interval optimization để lựa chọn tập entity tối ưu hơn. Ngoài ra, fuzzy matching cũng có thể được tích hợp nhằm xử lý các trường hợp entity bị viết tắt hoặc có sai lệch nhỏ về format.

Bên cạnh đó, hệ thống có thể được mở rộng thêm nhiều loại entity mới ngoài phạm vi hiện tại, chẳng hạn như tên tổ chức, mã số định danh, biển số xe, tài khoản ngân hàng hoặc thông tin học tập. Việc mở rộng tập entity sẽ giúp hệ thống phù hợp hơn với các bài toán anonymization thực tế trong doanh nghiệp hoặc các hệ thống lưu trữ dữ liệu lớn.

Một hướng phát triển quan trọng khác là kết hợp rule-based method với machine learning hoặc deep learning nhằm tận dụng ưu điểm của cả hai phương pháp. Rule-based approach có tính giải thích cao và hoạt động tốt với các mẫu cố định, trong khi machine learning có khả năng học được các pattern phức tạp và tổng quát hóa tốt hơn trên dữ liệu đa dạng. Việc kết hợp hybrid approach có thể giúp cải thiện đáng kể hiệu năng của hệ thống NER trong các ngữ cảnh thực tế.

Ngoài phần xử lý backend, project cũng có thể được mở rộng bằng cách xây dựng giao diện web local để trực quan hóa kết quả nhận diện entity. Người dùng có thể nhập văn bản trực tiếp lên website, sau đó hệ thống sẽ highlight các thực thể được phát hiện và hiển thị phiên bản đã được ẩn danh. Điều này giúp project dễ demo hơn và tăng tính ứng dụng thực tế của hệ thống.

Tóm lại, dù vẫn còn nhiều hạn chế, project đã xây dựng được một nền tảng tương đối hoàn chỉnh cho bài toán nhận diện và ẩn danh thông tin cá nhân bằng phương pháp rule-based. Các hướng mở rộng trong tương lai không chỉ giúp cải thiện độ chính xác của hệ thống mà còn mở ra khả năng ứng dụng rộng hơn trong các bài toán NLP và bảo vệ dữ liệu cá nhân trong thời đại AI.



Tài liệu

- [1] A. A. Awan, “What is Named Entity Recognition (NER)? Methods, Use Cases, and Challenges,” <https://www.datacamp.com/blog/what-is-named-entity-recognition-ner>, 2023, accessed: May 19, 2026.
- [2] Ho Chi Minh City University of Technology, “Regular Expressions,” lecture slide, Section 2.3, Definition 2.3.